The Dutch Merchant Problem: A Complete Algorithmic Analysis

Abstract

This document presents a comprehensive analysis of the Dutch Merchant Problem, a combinatorial optimization problem that combines routing decisions with transaction optimization under multiple constraints. The work covers four main phases: (1) formal problem definition and mathematical modeling, (2) computational complexity analysis establishing NP-Completeness via reduction from Prize-Collecting TSP, (3) design and implementation of both exact brute force algorithms and efficient heuristic solutions, and (4) extensive experimental evaluation comparing solution quality and performance. The exact algorithms (pure brute force, pruned brute force, and hybrid brute force with dynamic programming) serve as optimal baselines for small instances ($n \leq 11$), while the two-phase hybrid algorithm (beam search route generation with multi-dimensional dynamic programming) efficiently handles medium-to-large instances ($15 \leq n \leq 30$) with multiple item types and k-unit operations. Experimental results demonstrate exponential growth in exact methods, validate the efficiency of the heuristic approach, and provide insights into algorithm behavior across different instance characteristics.

1 Introduction

The Dutch Merchant Problem models a strategic maritime trading expedition where a captain must plan both the route through a network of ports and the sequence of buy/sell transactions at each port to maximize final capital. The problem combines elements of routing optimization (similar to the Traveling Salesman Problem) with inventory management and financial planning under multiple constraints: limited cargo capacity, capital requirements, and time restrictions.

This work presents a complete algorithmic analysis following a structured four-phase approach:

1. **Problem Formalization**: Translation of the informal problem description into a precise mathematical model with well-defined input structures, constraints, and objective function.

2. **Computational Complexity Analysis**: Formal proof that the problem is NP-Complete via polynomial-time reduction from the Prize-Collecting Traveling Salesman Problem.

3. **Algorithmic Solutions Design**: Development of both exact algorithms (for establishing optimal baselines) and efficient heuristic solutions (for practical scalability).

4. **Implementation and Experimental Analysis**: Comprehensive empirical evaluation comparing solution quality, execution times, and identifying practical limits for each approach.

The remainder of this document is organized as follows: Section 2 presents the problem formalization, Section 3 establishes NP-Completeness, Section 4 describes the algorithmic solutions, Section 5 presents experimental results, and Section 6 concludes with key findings and future directions.

2 Problem Formalization

This section presents the formal mathematical model of the Dutch Merchant Problem. For a more detailed discussion of the problem description and design decisions, see the brute force algorithms documentation [8].

2.1 Problem Description

The Dutch Merchant Problem is inspired by the historical context of the Dutch East India Company's maritime expeditions. A captain, starting from Amsterdam, must plan a commercial expedition that visits a subset of ports and returns to the origin, maximizing the final capital while respecting operational constraints.

2.2 Mathematical Model

An instance of the Dutch Merchant Problem is defined by the tuple $(G, r, \mathcal{M}, \mathcal{P}, B, T_{max}, f, k)$ where:

- $G = (V, E)$ is a complete, undirected graph representing the port network. $V = \{v_0, v_1, ..., v_n\}$, where v_0 is the origin port (Amsterdam) and $v_1, ..., v_n$ are intermediate ports.
- $r \in \mathbb{R}^+$ is the initial capital provided by investors.
- $\mathcal{M} = \{m_1, m_2, ..., m_{|\mathcal{M}|}\}$ is the set of $m = |\mathcal{M}|$ item types (commodities) available at ports.
- $\mathcal{P} = \{(c^{buy}_{v,m}, c^{sell}_{v,m}) \mid v \in V \setminus \{v_0\}, m \in \mathcal{M}\}$ defines the purchase price $c^{buy}_{v,m}$ and sale price $c^{sell}_{v,m}$ for each item type m at each port v.
- $B \in \mathbb{N}$ is the maximum cargo capacity of the ship (total units across all item types).
- $T_{max} \in \mathbb{R}^+$ is the maximum total time allowed for the expedition.
- $f : E \to \mathbb{R}^+ \times \mathbb{R}^+$ assigns to each edge $e = (u, v)$ a travel time t_{uv} and an operational cost c_{uv}.
- $k \in \mathbb{N}$ is the maximum number of units that can be bought or sold in a single operation at a port (operation limit).

2.3 Decision Space

At each port $v \in V \setminus \{v_0\}$, for each item type $m \in \mathcal{M}$, the captain can choose:

- **Buy operation**: Purchase j units where $j \in \{0, 1, 2, ..., k\}$ (where $j = 0$ represents doing nothing for this item)
- **Sell operation**: Sell j units where $j \in \{1, 2, ..., k\}$ (if sufficient inventory exists)

The decision space per port is $O((k + 1)^m)$ when considering all item types simultaneously.

2.4 Solution Structure

A solution consists of:

1. A cycle $(v_0, v_{i_1}, v_{i_2}, ..., v_{i_l}, v_0)$ visiting a subset of ports, where each intermediate port appears at most once.
2. For each visited port v_{i_j}, a sequence of decisions specifying buy/sell operations for each item type.

2.5 Feasibility Constraints

A solution is **feasible** if it satisfies all of the following constraints:

1. **Capacity constraint**: At no point does the total cargo load exceed B:

$$\sum_{m \in \mathcal{M}} \text{load}_m \leq B \quad \text{at all times}$$

2. **Capital constraint**: After each operation and travel, capital remains non-negative:

$$c_{current} \geq 0$$

3. **Time constraint**: Total time (travel + operations) does not exceed T_{max}:

$$\sum_{e \in \text{tour}} t_e + \sum_{\text{ports}} \text{operation\_time} \leq T_{max}$$

4. **Inventory constraint**: Cannot sell more units than currently carried:

$$\text{sell\_quantity}_m \leq \text{current\_load}_m \quad \forall m \in \mathcal{M}$$

2.6 Objective Function

The objective is to maximize the final capital upon returning to Amsterdam (v_0):

$$\max \left\{ r + \sum_{\text{sales}} c_{v,m}^{sell} \cdot \text{units\_sold} - \sum_{\text{purchases}} c_{v,m}^{buy} \cdot \text{units\_bought} - \sum_{e \in \text{tour}} c_e \right\}$$

2.7 Problem Variants

For exact algorithms, we consider a simplified variant with:

- Single commodity type per port ($|\mathcal{M}| = 1$)
- Binary operations ($k = 1$): buy 1 unit, sell 1 unit, or do nothing
- Small capacity ($B = 3 - 4$)

For efficient algorithms, we extend to:

- Multiple item types ($|\mathcal{M}| \geq 1$)

- k-unit operations ($k \geq 1$)
- Larger capacity and instance sizes

3 Computational Complexity Analysis

This section establishes that the Dutch Merchant Problem is NP-Complete by demonstrating both NP-hardness (via reduction from Prize-Collecting TSP) and membership in NP. The reduction proof follows the standard approach for establishing NP-hardness [3]. For a more detailed exposition of the reduction and complexity preservation arguments, see the dedicated complexity analysis document [9].

3.1 NP-Hardness via Reduction from Prize-Collecting TSP

3.1.1 Prize-Collecting TSP Definition

An instance of the PRIZECOLLECTINGTSP (PCTSP) problem is given by $(H, s, \{\pi_u\}_{u \in U}, \{\omega_{ij}\}_{(i,j) \in F})$, where:

- $H = (U, F)$ is a complete graph
- $s \in U$ is the root node
- $\pi_u \geq 0$ is the prize for visiting node u
- $\omega_{ij} \geq 0$ is the cost of traversing edge (i, j)

The goal is to find a cycle that starts and ends at s, visits a subset of nodes, and maximizes $\sum_{u \in S} \pi_u - \sum_{e \in \text{tour}} \omega_e$.

Theorem 1 (NP-Hardness of PCTSP): PRIZECOLLECTINGTSP is NP-hard.

Proof: The Prize-Collecting TSP is a well-known generalization of the classic Traveling Salesman Problem (TSP) [5]. To see that PCTSP is NP-hard, consider a TSP instance with n cities. We can transform it into a PCTSP instance by:

1. Setting the root node s to be any city in the TSP instance
2. Assigning a prize $\pi_u = M$ to each city $u \neq s$, where M is a value greater than the sum of all edge costs in the TSP instance
3. Using the same edge costs ω_{ij} as in the TSP instance

In this transformed instance, any optimal PCTSP solution must visit all cities (since the prize M for each city exceeds any possible cost), and the objective reduces to minimizing the total tour cost, which is exactly the TSP objective. Since TSP is NP-hard [5], and we have a polynomial-time reduction from TSP to PCTSP, it follows that PCTSP is NP-hard. The NP-hardness of PCTSP is also well-established in the literature on prize-collecting routing problems [11]. □

3.1.2 Restricted Dutch Merchant Problem

For the reduction, we define RESTRICTED-DMP, a simplified version of the Dutch Merchant Problem with:

1. Single commodity: $|\mathcal{M}| = 1$
2. Fixed prices: $c_v^{buy} = 0$ and $c_v^{sell} = p_v \geq 0$ for every port v
3. Unlimited capacity: $B = \infty$
4. No intermediate capital constraint
5. Unlimited time: $T_{max} = \infty$

An instance of RESTRICTED-DMP is defined by $(G, v_0, \{p_v\}_{v \in V}, \{c_e\}_{e \in E})$, and its objective is to find a cycle starting and ending at v_0, visiting a subset $S \subseteq V \setminus \{v_0\}$, that maximizes $\sum_{v \in S} p_v - \sum_{e \in \text{tour}} c_e$.

3.1.3 Reduction Construction

Given an arbitrary instance $I_P = (H, s, \{\pi_u\}, \{\omega_{ij}\})$ of PCTSP, we construct an instance $I_C = (G, v_0, \{p_v\}, \{c_e\})$ of RESTRICTED-DMP in polynomial time:

1. For each node $u \in U$ in H, create a corresponding port $v \in V$ in G. Map root node s to origin port v_0.
2. For each edge (i, j) in H with cost ω_{ij}, define $c_{v_i v_j} = \omega_{ij}$ in G.
3. For each port v corresponding to node $u \neq s$, define $p_v = \pi_u$.

This construction runs in $O(|U|^2)$ time.

3.1.4 Solution Equivalence

Lemma 1: There exists a tour in I_P with value $\geq K$ if and only if there exists a tour in I_C with final capital $\geq r + K$.

Proof: We establish a bijective correspondence between feasible tours of I_P and I_C that preserves the objective function.

Forward Direction ($\Rightarrow$): Suppose there exists a tour $T_P = (s, u_{i_1}, ..., u_{i_m}, s)$ in I_P with value $V_P = \sum_{k=1}^{m} \pi_{u_{i_k}} - \sum_{e \in T_P} \omega_e \geq K$.

We construct the corresponding tour $T_C = (v_0, v_{i_1}, ..., v_{i_m}, v_0)$ in I_C as follows:

1. Start at v_0 (which corresponds to s)
2. Visit ports $v_{i_1}, ..., v_{i_m}$ in the same order as nodes $u_{i_1}, ..., u_{i_m}$ in T_P
3. Return to v_0

By the construction of I_C:

- Each port v_{i_k} corresponds to node u_{i_k}, so $p_{v_{i_k}} = \pi_{u_{i_k}}$
- Each edge $(v_{i_k}, v_{i_{k+1}})$ in T_C corresponds to edge $(u_{i_k}, u_{i_{k+1}})$ in T_P, so $c_{v_{i_k} v_{i_{k+1}}} = \omega_{u_{i_k} u_{i_{k+1}}}$
- The edge from v_{i_m} to v_0 has cost $c_{v_{i_m} v_0} = \omega_{u_{i_m} s}$

In RESTRICTED-DMP, visiting port v automatically yields profit p_v with no capacity or capital constraints (since $B = \infty$ and there are no intermediate capital constraints). Therefore, the final capital in I_C is:

$$C_C = r + \sum_{k=1}^{m} p_{v_{i_k}} - \sum_{e \in T_C} c_e = r + \sum_{k=1}^{m} \pi_{u_{i_k}} - \sum_{e \in T_P} \omega_e = r + V_P \geq r + K$$

Reverse Direction ($\Leftarrow$): Suppose there exists a tour $T_C = (v_0, v_{i_1}, ..., v_{i_m}, v_0)$ in I_C with final capital $C_C = r + \sum_{k=1}^{m} p_{v_{i_k}} - \sum_{e \in T_C} c_e \geq r + K$.

We construct the corresponding tour $T_P = (s, u_{i_1}, ..., u_{i_m}, s)$ in I_P by mapping each port v_{i_k} back to its corresponding node u_{i_k}. By the same correspondence argument, the value of T_P is:

$$V_P = \sum_{k=1}^{m} \pi_{u_{i_k}} - \sum_{e \in T_P} \omega_e = \sum_{k=1}^{m} p_{v_{i_k}} - \sum_{e \in T_C} c_e = C_C - r \geq K$$

Optimality Preservation: Since r is a constant, maximizing C_C in I_C is equivalent to maximizing V_P in I_P. Therefore, an optimal solution to I_P corresponds to an optimal solution to I_C, and vice versa. □

Theorem 2: RESTRICTED-DMP is NP-hard.

Proof: By Theorem 1, PCTSP is NP-hard. By Lemma 1, we have established a polynomial-time reduction from PCTSP to RESTRICTED-DMP that preserves optimality. The reduction construction takes $O(|U|^2)$ time, which is polynomial in the input size. Since we can solve RESTRICTED-DMP if and only if we can solve PCTSP, and PCTSP is NP-hard, it follows that RESTRICTED-DMP is NP-hard. □

3.1.5 Complexity Preservation

Theorem 3: The general DUTCHMERCHANT problem is NP-hard.

Proof: We demonstrate that reintroducing each eliminated constraint does not reduce complexity. The key insight is that RESTRICTED-DMP is a **special case** of the general DUTCHMERCHANT problem under specific parameter settings. Since a special case of a problem cannot be harder than the general problem, if RESTRICTED-DMP is NP-hard, then the general problem must also be NP-hard [3].

Let Π_{orig} denote the general DUTCHMERCHANT problem and Π_{restr} denote RESTRICTED-DMP. We show that for each constraint eliminated in Π_{restr}, there exists a configuration of Π_{orig} that makes it equivalent to Π_{restr}:

1. Multiple Commodities ($|\mathcal{M}| \geq 1$): In Π_{restr}, $|\mathcal{M}| = 1$. Consider an instance of Π_{orig} with $|\mathcal{M}| = k > 1$ where:

- Purchase prices for commodities $2, ..., k$ are set to a value higher than any possible profit (e.g., $c^{buy}_{v,m} > \max_{u,w} c^{sell}_{u,w}$ for all $m \geq 2$)
- Sale prices for commodity 1 are identical to the profit values in Π_{restr}: $c^{sell}_{v,1} = p_v$ and $c^{buy}_{v,1} = 0$
- Capacity B is set sufficiently large to carry only commodity 1

Any optimal solution will ignore commodities $2, ..., k$ and behave identically to Π_{restr}. Therefore, Π_{restr} is a special case of Π_{orig} with multiple commodities.

2. Finite Cargo Capacity ($B \in \mathbb{N}$): In Π_{restr}, $B = \infty$. Setting B to a sufficiently large value in Π_{orig} (e.g., $B \geq \sum_{v \in V} 1$) replicates the unlimited capacity condition, as the limit will never be reached with a single unit-profit commodity. Reducing B introduces a knapsack-like constraint, which is NP-hard [5]. Adding an NP-hard constraint cannot simplify a problem already proven NP-hard.

3. Intermediate Financial Constraint (Non-Negative Capital): In Π_{restr}, this constraint is omitted. In Π_{orig}, if the initial capital r is greater than the maximum possible sum of travel costs, the constraint never activates, replicating Π_{restr}. Reducing r introduces a

dynamic financial state that must be verified at each port, increasing the complexity of the state space, not reducing it.

4. Strict Time Limit ($T_{max} \in \mathbb{R}^+$): In Π_{restr}, $T_{max} = \infty$. Setting T_{max} larger than the duration of the longest possible tour in Π_{orig} renders the constraint inactive, replicating Π_{restr}. Imposing a strict time limit transforms the problem into a variant of the Orienteering Problem, which is also NP-hard [3].

5. Complex Buy/Sell Decisions: In Π_{restr}, profit is automatic ($c_v^{buy} = 0, c_v^{sell} = p_v$). In Π_{orig}, fixing prices identically and defining that profit is obtained upon port arrival (without capacity consumption) simulates Π_{restr}. Allowing variable prices and strategic decisions introduces an arbitrage and inventory management problem, which is a strict and more difficult generalization.

Conclusion: For each constraint R_i, there exists a configuration of Π_{orig} that makes it equivalent to Π_{restr}. Any adjustment that genuinely activates R_i (e.g., reducing B, reducing T_{max}) restricts the feasible solution space and/or adds an NP-hard optimization dimension. Since Π_{restr} is NP-hard (Theorem 2) and Π_{restr} is a special case of Π_{orig}, it follows that Π_{orig} is NP-hard. □

3.2 Membership in NP

To establish NP-Completeness, we must prove both NP-hardness (established in the previous subsection) and membership in NP. A problem belongs to NP if, given a proposed solution (certificate), we can verify its correctness in polynomial time [3].

Lemma 2: The Dutch Merchant Problem belongs to the class NP.

Proof: We demonstrate that there exists a polynomial-time verifier for the Dutch Merchant Problem.

Certificate Definition: A certificate C for an instance of the Dutch Merchant Problem consists of:

1. A tour $T = (v_0, v_{i_1}, v_{i_2}, ..., v_{i_m}, v_0)$ specifying the sequence of ports visited, where v_0 is Amsterdam and $v_{i_1}, ..., v_{i_m}$ are intermediate ports (each appearing at most once)
2. For each visited port v_{i_j}, a list of buy/sell operations specifying:
    - For each item type $m \in \mathcal{M}$, the number of units bought (non-negative integer $\leq k$)
    - For each item type $m \in \mathcal{M}$, the number of units sold (non-negative integer $\leq k$, and cannot exceed current inventory)

The size of this certificate is polynomially bounded: the tour has at most $|V|$ nodes, and operations at each port are specified for $|\mathcal{M}|$ commodities, with each operation requiring $O(1)$ space. Therefore, $|C| = O(|V| \cdot |\mathcal{M}|)$, which is polynomial in the input size.

Verifier Algorithm: We now describe a polynomial-time verifier $V(I, C)$ that checks whether certificate C represents a feasible solution with objective value at least K (for the decision problem: "Does there exist a solution with final capital $\geq K$?").

The verifier performs the following checks:

Step 1: Tour Validity ($O(|V|)$):

- Verify that T starts and ends at v_0 (Amsterdam)
- Verify that no intermediate port appears more than once (check that all v_{i_j} for $j = 1, ..., m$ are distinct)
- Verify that all ports in T belong to V

This requires a single pass through the tour, checking each port against a set of visited ports. Time complexity: $O(|V|)$.

Step 2: Capacity Compliance ($O(m \cdot |\mathcal{M}|)$):

- Initialize load vector $\vec{\ell} = (0, ..., 0)$ of dimension $|\mathcal{M}|$
- For each port v_{i_j} in order:
  - Update load: $\ell_m \leftarrow \ell_m + \text{bought}_m - \text{sold}_m$ for each $m \in \mathcal{M}$
  - Verify: $\sum_{m \in \mathcal{M}} \ell_m \leq B$

This requires iterating through m ports and, for each port, updating $|\mathcal{M}|$ load values and checking the capacity constraint. Time complexity: $O(m \cdot |\mathcal{M}|)$.

Step 3: Financial Constraint Compliance ($O(m \cdot |\mathcal{M}|)$):

- Initialize capital $c = r$ (initial capital)
- For each port v_{i_j} in order:
  - Subtract travel cost: $c \leftarrow c - c_{v_{i_{j-1}}, v_{i_j}}$ (where $v_{i_0} = v_0$)
  - Add sale revenue: $c \leftarrow c + \sum_{m \in \mathcal{M}} c^{sell}_{v_{i_j}, m} \cdot \text{sold}_m$
  - Subtract purchase costs: $c \leftarrow c - \sum_{m \in \mathcal{M}} c^{buy}_{v_{i_j}, m} \cdot \text{bought}_m$
  - Verify: $c \geq 0$ (capital remains non-negative)
- After returning to Amsterdam: $c \leftarrow c - c_{v_{i_m}, v_0}$

- Verify: $c \geq 0$

This requires iterating through m ports and, for each port, performing $O(|\mathcal{M}|)$ arithmetic operations. Time complexity: $O(m \cdot |\mathcal{M}|)$.

Step 4: Inventory Constraint Compliance ($O(m \cdot |\mathcal{M}|)$):

- Maintain load vector $\vec{\ell}$ as in Step 2
- For each port v_{i_j} and each item type m:
  - Before applying operations, verify: sold$_m \leq \ell_m$ (cannot sell more than currently carried)

This is performed alongside Step 2, so it adds no additional asymptotic complexity. Time complexity: $O(m \cdot |\mathcal{M}|)$.

Step 5: Time Limit Compliance ($O(m)$):

- Initialize total time $t = 0$
- For each edge $(v_{i_j}, v_{i_{j+1}})$ in the tour (including (v_0, v_{i_1}) and (v_{i_m}, v_0)):
  - Add travel time: $t \leftarrow t + t_{v_{i_j}, v_{i_{j+1}}}$
- For each port v_{i_j}:
  - Add operation time: $t \leftarrow t +$ operation_time(v_{i_j}) (e.g., 1 unit per buy/sell operation)
- Verify: $t \leq T_{max}$

This requires iterating through $m + 1$ edges and m ports. Time complexity: $O(m)$.

Step 6: Objective Calculation ($O(1)$):

- The final capital c after Step 3 is the objective value
- Return `true` if $c \geq K$, `false` otherwise

Time complexity: $O(1)$.

Total Verification Time: The total time complexity is:

$$O(|V|) + O(m \cdot |\mathcal{M}|) + O(m \cdot |\mathcal{M}|) + O(m \cdot |\mathcal{M}|) + O(m) + O(1) = O(|V| + m \cdot |\mathcal{M}|)$$

Since $m \leq |V|$ (the tour visits at most all ports), this is $O(|V| \cdot |\mathcal{M}|)$, which is polynomial in the input size. The input size includes $|V|$, $|E| = O(|V|^2)$, and $|\mathcal{M}|$, so $O(|V| \cdot |\mathcal{M}|)$ is indeed polynomial.

Conclusion: There exists a polynomial-time verifier $V(I, C)$ that can check in $O(|V| \cdot |\mathcal{M}|)$ time whether a certificate C represents a feasible solution with objective value at least K. Therefore, the Dutch Merchant Problem belongs to the class NP. □

3.3 NP-Completeness Conclusion

Theorem 4: The decision problem associated with DUTCHMERCHANT is NP-Complete.

Proof: By Theorem 3, the problem is NP-hard. By Lemma 2, it belongs to NP. Therefore, it is NP-Complete. □

This classification formally justifies the use of non-exact algorithmic techniques (heuristics, approximations) for practical problem solving.

4 Algorithmic Solutions Design

Given the NP-Completeness of the problem, we develop both exact algorithms (for small instances) and efficient heuristic solutions (for larger instances). The exact algorithms are detailed in the brute force algorithms documentation [8], while the efficient two-phase hybrid approach is described in the efficient solution report [10].

4.1 Exact Algorithms

We present three progressively more efficient exact algorithms that guarantee optimal solutions for small instances.

4.1.1 Pure Brute Force Algorithm

The pure brute force algorithm serves as the baseline exact method, explicitly enumerating all possible solutions [2].

Algorithm Description:

1. Generate all subsets of visitable ports (excluding Amsterdam)
2. For each subset, generate all possible visit orderings (permutations)
3. For each route, generate all possible decision sequences (buy/sell/nothing at each port)
4. Evaluate each complete solution for feasibility and objective value

5. Return the solution with maximum final capital

Complexity Analysis: The algorithm explores [2]:

- All subsets: $\sum_{k=0}^{n-1} \binom{n-1}{k} = 2^{n-1}$ subsets
- All permutations: $k!$ for each subset of size k [6]
- All decision sequences: 3^k for a route visiting k ports

Total complexity: $T(n) = \sum_{k=0}^{n-1} \binom{n-1}{k} \cdot k! \cdot 3^k = O(n! \cdot 3^n)$

Practical Limits: Due to exponential growth, this algorithm is only practical for instances with $n \leq 8$ ports within a 200-second timeout.

4.1.2 Brute Force with Pruning

The pruned brute force algorithm maintains exhaustive enumeration but incorporates backtracking and early termination to eliminate infeasible branches. This approach uses depth-first search with branch-and-bound techniques [4, 7].

Pruning Techniques:

1. **Feasibility Pruning**: Immediately abandons branches where capital becomes negative, load exceeds capacity, or time exceeds T_{max}
2. **Optimistic Bound Pruning**: Estimates maximum possible profit from remaining ports; prunes if optimistic bound cannot improve current best
3. **Time-Based Route Pruning**: Before exploring a route subset, estimates minimum tour time; discards routes that cannot be completed within T_{max}

Complexity Analysis: Worst-case complexity remains $O(n! \cdot 3^n)$, but effective complexity is $T(n) = O(\alpha \cdot n! \cdot 3^n)$ where $\alpha < 1$ is the pruning factor. Empirical analysis shows pruning rates of 20.4% for $n = 5$, increasing to 46.0% for $n = 11$.

Practical Limits: Extends practical limits to $n \leq 9$ ports (times out at $n = 10$).

4.1.3 Hybrid Algorithm: Brute Force + Dynamic Programming

The hybrid algorithm separates route selection (combinatorial) from transaction optimization (sequential decision-making with optimal substructure). This decomposition exploits the optimal substructure property of sequential decision problems [1].

Key Insight: For a fixed route, the transaction optimization subproblem exhibits optimal substructure and can be solved efficiently using dynamic programming.

Dynamic Programming Formulation:
For a fixed route $R = [v_0, v_{i_1}, ..., v_{i_m}, v_0]$ with m intermediate ports:

- **State Space**: (i, c, ℓ) where:
  - $i \in \{0, 1, ..., m\}$: current port index
  - c: current capital (may be discretized)
  - $\ell \in \{0, 1, ..., B\}$: current load
- **Value Function**: $V(i, c, \ell)$ = maximum final capital achievable starting from port i with capital c and load ℓ
- **Recurrence Relation**:

$$V(i, c, \ell) = \max_{d \in \{0,1,2\}} \begin{cases} V(i+1, c', \ell') & \text{if } d = 0 \text{ (do nothing)} \\ V(i+1, c'', \ell+1) & \text{if } d = 1 \text{ (buy), feasible} \\ V(i+1, c''', \ell-1) & \text{if } d = 2 \text{ (sell), feasible} \end{cases}$$

  where c', c'', c''' are capital levels after travel costs and operations, and feasibility checks ensure capital remains non-negative and load stays within $[0, B]$.

Complexity Analysis:

$$T(n) = O(n! \cdot B \cdot K)$$

where B (capacity) and K (discretized capital levels) are fixed parameters, not growing with n. This replaces the exponential 3^n transaction enumeration with polynomial $B \cdot K$ DP computation.

Practical Limits: Can solve instances with $n \leq 11$ ports within the 200-second timeout.

4.2 Efficient Algorithm: Two-Phase Hybrid Approach

For medium-to-large instances ($15 \leq n \leq 30$), we design a two-phase hybrid algorithm that separates route generation from transaction optimization.

4.2.1 Algorithm Architecture

1. **Phase 1**: Generate promising routes using PCTSP-inspired heuristics (beam search)
2. **Phase 2**: For each route, solve the transaction optimization problem using multi-dimensional dynamic programming

4.2.2 Phase 1: Route Generation with Beam Search

Beam search maintains a beam (set) of the best partial routes at each depth, expanding only the most promising candidates.

PCTSP Bound Calculation: For a route R, calculate an upper bound:

$$\text{bound}(R) = \sum_{v \in R} \pi_v - \sum_{e \in R} c_e$$

where $\pi_v = \max_{m \in \mathcal{M}} \left\{ (c_{v,m}^{sell} - c_{v,m}^{buy}) \cdot \min(k, B) \right\}$ is the maximum profit per port.

Beam Search Algorithm:

1. Initialize beam with route $[v_0]$
2. For each depth level:
   (a) Expand all routes in beam to unvisited ports
   (b) Calculate PCTSP bounds for all candidates
   (c) Keep top K candidates by bound (beam width)
   (d) Add complete routes to final set if time-feasible
3. Return routes sorted by bound (descending)

Complexity: $O(K \times n^3)$ where K is the beam width (controlled parameter).

4.2.3 Phase 2: Multi-Dimensional Transaction DP

For a fixed route $R = [v_0, v_{i_1}, ..., v_{i_L}, v_0]$ of length $L + 2$:

State Representation: $(i, c, \vec{\ell})$ where:

- $i \in \{0, 1, ..., L\}$: Port index
- $c \in \mathbb{R}^+$: Capital level (may be discretized)
- $\vec{\ell} = (\ell_1, \ell_2, ..., \ell_m) \in [0, B]^m$: Load vector for m item types

DP Recurrence: Process ports sequentially, for each state $(c, \vec{\ell})$ at port i, generate all feasible actions (do nothing, buy, sell) for all items, update states for port $i + 1$.

Optimization Strategies:

1. **Capital Discretization**: Group capital values into bands of width $\Delta = \max(1.0, r/100)$

2. **Sparse Representation**: Use dictionaries instead of dense arrays for sparse state spaces

3. **Early Pruning**: Skip DP computation if route's PCTSP bound is worse than current best solution

Complexity: For each route of length L: $O(L \times m \times k \times B^m \times K_{capital})$

4.2.4 Overall Algorithm

Function TwoPhaseHybridSolve(instance, timeout, beam_width K):

1. Phase 1: Generate promising routes using beam search
2. Phase 2: For each route (until timeout):
   (a) Solve transaction optimization using DP
   (b) Update best solution if improved
3. Return best solution

Overall Complexity: $O(K \times n^3 + K \times n \times m \times k \times B^m \times K_{capital})$

Unlike brute force's $O(n! \times (k+1)^{m \cdot n})$, this is polynomial in K (controlled) and fixed parameters m, k, B, and $K_{capital}$.

5 Implementation and Experimental Analysis

This section presents comprehensive experimental results comparing all algorithmic approaches. Detailed experimental data and additional analysis can be found in the brute force algorithms documentation [8] and the efficient solution report [10].

5.1 Experimental Setup

5.1.1 Instance Generation

Test instances were generated with the following characteristics:

Small Instances (for Exact Algorithms):

- Ports: $n \in \{3, 4, 5, 6, 7, 8, 9, 10, 11\}$ (including Amsterdam)
- Single commodity type ($|\mathcal{M}| = 1$)

- Binary operations ($k = 1$)
- Travel costs: Random integers in $[2, 15]$
- Travel times: Random integers in $[1, 8]$
- Purchase prices: Random integers in $[8, 25]$ per port
- Sale prices: Purchase price + random margin in $[12, 25]$ per port
- Initial capital: $r = 50 - 150$ (scaling with instance size)
- Capacity: $B = 3 - 4$
- Maximum time: $T_{max} = 15 + 8(n - 2)$

Medium/Large Instances (for Efficient Algorithm):

- Ports: $n \in \{15, 20, 25, 30\}$
- Item types: $m = 2$ items per port
- Operations: $k = 2$ units per buy/sell operation
- Capacity: $B = \min(5, n - 1)$
- Time limit: $T_{max} = 20 + (n - 2) \times 10$
- Initial capital: $r = 100 + (n - 3) \times 20$

Guarantees: Instance generators ensure at least one profitable route exists, feasible solutions exist, and non-trivial solutions (not just $[Amsterdam, Amsterdam]$).

Environment: All tests were run on a standard desktop computer. Random seeds were fixed for reproducibility. Timeout parameter: 200 seconds.

5.2 Exact Algorithms Results

5.2.1 Execution Time Comparison

Table 1 presents execution times for all three exact algorithms.

Observations:

- Pure brute force becomes impractical for $n \geq 9$ (times out at 200s)
- Pruning provides 2-4$\times$ speedup, extending practical limit to $n \leq 9$

Table 1: Execution Time Comparison (seconds)

n	Pure BF	Pruned BF	Hybrid	Speedup (Pruned)	Speedup (Hybrid)
3	0.0003	0.0001	0.0003	2.1×	1.0×
4	0.0004	0.0004	0.0004	1.1×	1.0×
5	0.0067	0.0021	0.0017	3.1×	3.8×
6	0.1018	0.0308	0.0312	3.3×	3.3×
7	1.0368	0.2730	0.1856	3.8×	5.6×
8	21.2869	5.0484	0.3884	4.2×	54.8×
9	> 200 (TO)	90.7548	9.7152	$> 2.2\times$	$> 20.6\times$
10	N/A	> 200 (TO)	5.1573	N/A	N/A
11	N/A	N/A	31.9570	N/A	N/A
12	N/A	N/A	> 200 (TO)	N/A	N/A

- Hybrid approach achieves 1-55× speedup, solving instances up to $n = 11$
- Growth is clearly exponential for all algorithms, but with different bases

5.2.2 Pruning Effectiveness

Table 2 shows pruning effectiveness breakdown for the pruned brute force algorithm.

Table 2: Pruning Effectiveness Breakdown

n	Total Pruned (%)	By Time (%)	By Bound (%)	By Capacity (%)	By Capital (%)
5	20.4	0.0	11.0	85.8	3.2
7	25.5	0.0	40.4	59.6	0.0
9	31.7	0.0	52.0	48.0	0.0
11	46.0	0.0	61.9	38.1	0.0

Observations:

- Pruning rate increases with instance size: 20.4% for $n = 5$, rising to 46.0% for $n = 11$
- Capacity-based pruning dominates for smaller instances (85.8% for $n = 5$)
- Bound-based pruning becomes dominant for larger instances (61.9% for $n = 11$)

5.2.3 Quality Verification

All three exact algorithms find the same optimal solutions when they complete execution on the same instances. Verified by:

- Running all algorithms on instances with $n \in \{3, 4, 5\}$
- Comparing final capital values: all algorithms found identical optimal capitals
- Verifying that solutions satisfy all constraints

This confirms that all algorithms are exact methods, differing only in efficiency, not solution quality.

5.3 Efficient Algorithm Results

5.3.1 Small Instance Comparison

For small single-commodity instances where exact algorithms are tractable:

- For $n = 2, 3, 4, 5, 6, 7$: The efficient algorithm matches the exact optimal capital within numerical tolerance
- For $n = 8$: The efficient algorithm finds a slightly lower capital than the exact optimum (182.0 vs. 184.0), while being roughly two orders of magnitude faster (0.33s vs. 22.0s)
- In all feasible cases: The efficient algorithm returns a final capital greater than or equal to the initial capital, and solutions pass all feasibility checks

5.3.2 Medium/Large Instance Performance

Table 3 presents execution results for larger instances.

Table 3: Algorithm Performance on Medium/Large Instances

n	Routes Generated	Routes Evaluated	Execution Time (s)	Final Capital	Timeout
15	1,141	1	0.43	593.0	No
20	2,569	1	0.96	847.2	No
25	4,424	1	1.78	1,071.6	No
30	5,429	52	61.34	1,297.2	No

Observations:

- Successfully handles instances up to $n = 30$ within reasonable time (under 62 seconds)

- Beam search effectively limits route exploration: For $n = 30$, only 5,429 routes generated (compared to $30! \approx 2.65 \times 10^{32}$ possible routes)
- Most routes are pruned by PCTSP bounds: For $n \leq 25$, only 1 route needed DP evaluation
- Execution time grows sub-exponentially with n
- Finds profitable solutions for all tested instances

5.3.3 Beam Width Sensitivity

For a representative instance with $n = 15$ ports, evaluating beam widths $\{50, 100, 200, 500\}$:

- Increasing beam width from 50 to 200: Modest time increase (13.5s to 14.7s) but improves capital from 435.4 to 438.8
- Further increasing to 500: No additional capital gains (remains 438.8) but increases time to 16.0s
- Beam width $K = 200$ provides the best trade-off between execution time and solution quality

5.4 Visualizations

Figure 1 shows exponential growth in execution time for exact algorithms on a logarithmic scale.

Figure 2 provides a direct comparison of execution times for instances where all exact algorithms completed.

Figure 3 shows the maximum solvable instance size for different time budgets.

5.5 Analysis of Behavior

5.5.1 When Each Algorithm Performs Best

Pure Brute Force:

- Best for: Very small instances ($n \leq 3$) where overhead is unnecessary
- Worst for: Any instance with $n \geq 5$ due to exponential explosion

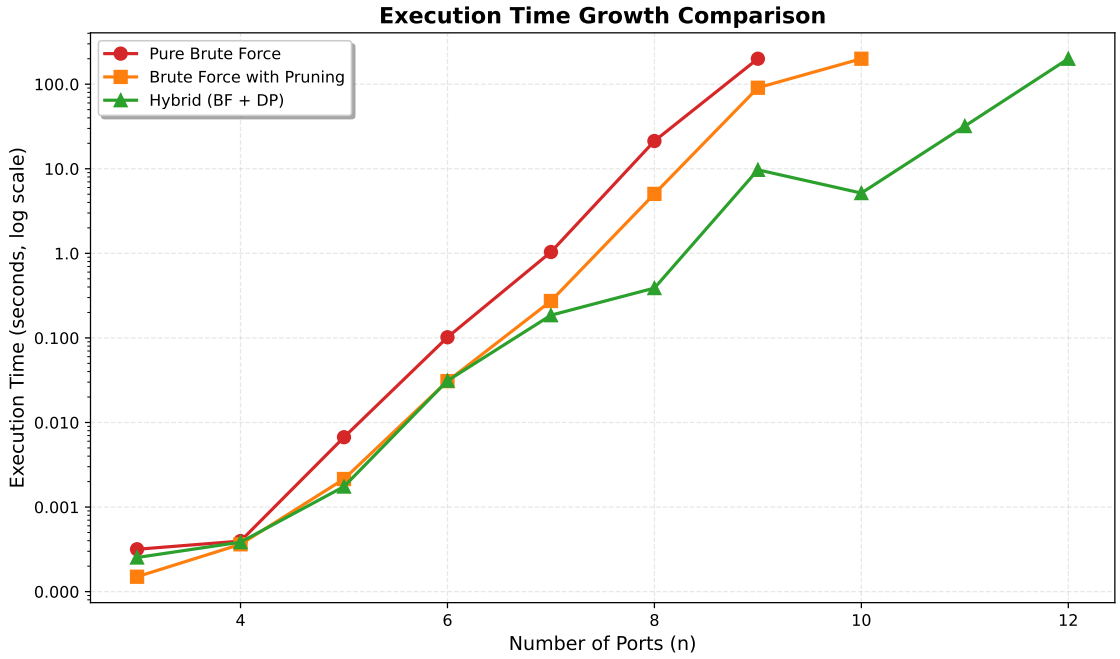

Figure 1: Execution Time Growth Comparison (Log Scale)

Brute Force with Pruning:

- Best for: Instances with tight constraints (low capital, tight time limits) where pruning is very effective
- Worst for: Instances with loose constraints where most branches are feasible

Hybrid (BF + DP):

- Best for: All instances $n \geq 4$ where it consistently outperforms other exact methods
- Worst for: Very small instances ($n \leq 3$) where overhead may not be justified

Two-Phase Hybrid:

- Best for: Medium-to-large instances ($n \geq 15$) where exact methods become intractable
- Provides near-optimal solutions for small instances and efficient solutions for large instances

5.5.2 Practical Limits

Based on experimental results with a 200-second timeout:

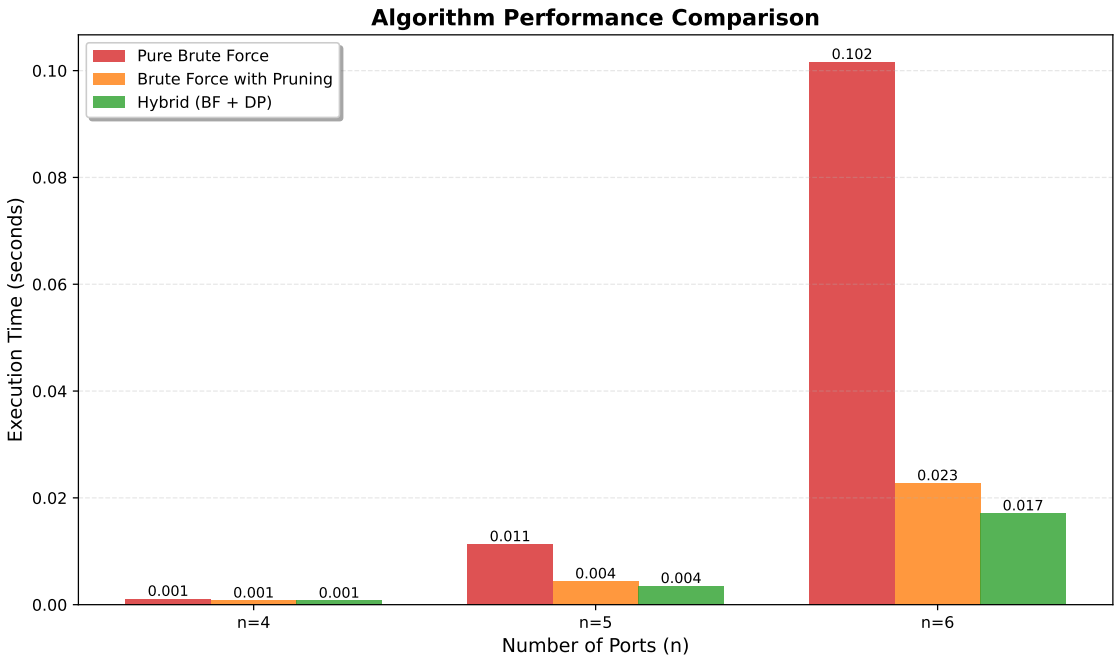

Figure 2: Algorithm Performance Comparison

- **Pure Brute Force**: Maximum $n = 8$ (completes in ∼21s), times out at $n = 9$
- **Pruned Brute Force**: Maximum $n = 9$ (completes in ∼91s), times out at $n = 10$
- **Hybrid**: Maximum $n = 11$ (completes in ∼32s), times out at $n = 12$
- **Two-Phase Hybrid**: Handles $n = 30$ within 62 seconds

For instances with $n > 11$, exact methods become impractical, justifying the need for heuristic approaches.

6 Conclusions

This work presents a comprehensive analysis of the Dutch Merchant Problem, covering problem formalization, complexity analysis, algorithmic design, and experimental evaluation.

6.1 Key Contributions

1. **Problem Formalization**: Established a precise mathematical model with well-defined input structures, constraints, and objective function, supporting both simplified (single commodity) and extended (multiple items) variants.

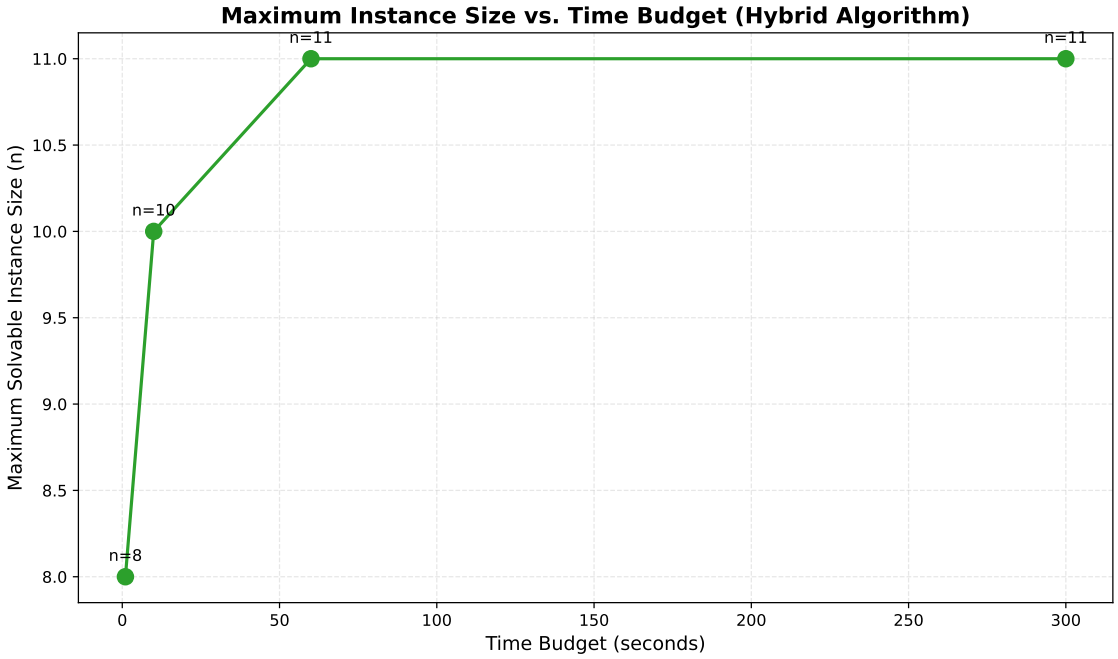

Figure 3: Maximum Instance Size vs. Time Budget (Hybrid Algorithm)

2. **Complexity Analysis**: Formally proved NP-Completeness via polynomial-time reduction from Prize-Collecting TSP, providing theoretical justification for heuristic approaches.

3. **Exact Algorithms**: Developed three progressively more efficient exact algorithms:

   - Pure brute force: Baseline optimal method for $n \leq 8$
   - Pruned brute force: 2-4$\times$ speedup, extends to $n \leq 9$
   - Hybrid (BF + DP): 1-55$\times$ speedup, extends to $n \leq 11$

4. **Efficient Algorithm**: Designed a two-phase hybrid algorithm combining beam search route generation with multi-dimensional dynamic programming, successfully handling instances up to $n = 30$ within reasonable time.

5. **Experimental Validation**: Comprehensive empirical analysis demonstrating:

   - Exponential growth in exact methods, confirming intractability
   - Effectiveness of pruning techniques (20.4% to 46.0% pruning rates)
   - Near-optimal solution quality for efficient algorithm on small instances
   - Scalability of two-phase approach for medium-to-large instances

6.2 Main Findings

- All exact algorithms guarantee optimal solutions, verified through cross-validation on instances with $n \in \{3, 4, 5\}$ where all found identical optimal capitals.
- Exponential growth is evident in all exact approaches, confirming the problem's intractability. Execution time grows from milliseconds ($n = 3$) to seconds ($n = 8$) to timeouts ($n \geq 9$ for Pure BF).
- The hybrid exact approach is the most efficient exact method, solving instances 2-3 ports larger than pure brute force while maintaining optimality guarantees.
- Pruning effectiveness increases with instance size: from 20.4% for $n = 5$ to 46.0% for $n = 11$. Capacity-based pruning dominates for smaller instances, while bound-based pruning becomes dominant for larger instances.
- The two-phase hybrid algorithm successfully addresses medium-to-large instances ($15 \leq n \leq 30$), achieving sub-exponential time growth and finding profitable solutions for all tested instances.
- For instances with $n > 11$, exact methods become impractical, justifying heuristic approaches. The two-phase hybrid algorithm provides an effective solution for these cases.

6.3 Future Work

Several directions for future research emerge from this study:

1. **Adaptive Beam Control**: Dynamically adjust the beam width based on the distribution of route bounds and intermediate DP results.
2. **Parallelization**: Exploit parallel hardware by evaluating multiple candidate routes concurrently in Phase 2.
3. **Enhanced Route Heuristics**: Integrate more advanced PCTSP approximation algorithms and local improvement procedures into Phase 1.
4. **State Space Refinement**: Further reduce the DP state space for large m or B through item grouping or decomposition techniques.
5. **Approximation Guarantees**: Develop theoretical approximation ratios for the two-phase hybrid algorithm.

6. **Multi-Objective Extensions**: Consider trade-offs between profit maximization and other objectives (e.g., risk minimization, route diversity).

6.4 Final Remarks

This work demonstrates that the Dutch Merchant Problem, while NP-Complete, can be effectively addressed through a combination of exact methods (for small instances) and efficient heuristic approaches (for larger instances). The experimental results validate the theoretical analysis and provide practical insights into algorithm behavior across different instance characteristics. The developed algorithms and analysis framework provide a solid foundation for further research and practical applications.

References

[1] Richard Bellman. Dynamic programming. *Princeton University Press*, 1957.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

[3] Michael R. Garey and David S. Johnson. *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman and Company, 1979.

[4] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Computer Algorithms*. Computer Science Press, 1978.

[5] Richard M. Karp. Reducibility among combinatorial problems. *Complexity of computer computations*, pages 85–103, 1972.

[6] Donald E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. Addison-Wesley Professional, 2011.

[7] Eugene L. Lawler and David E. Wood. Branch-and-bound methods: A survey. *Operations Research*, 14(4):699–719, 1966.

[8] Ariadna Velázquez Lia López. Brute force algorithms for the dutch merchant problem, 2025. Documentation/Brute_Force_Inform.pdf - Detailed description of exact algorithms.

[9] Ariadna Velázquez Lia López. Complexity analysis of the dutch merchant problem, 2025. Documentation/Reduction.pdf - Polynomial reduction from Prize-Collecting TSP.

[10] Ariadna Velázquez Lia López. Two-phase hybrid algorithm for the dutch merchant problem, 2025. Documentation/Efficient_Solution_Report.pdf - Detailed description of efficient algorithm.

[11] Alberto Marchetti-Spaccamela, Vincenzo Bonifaci, Stefano Leonardi, and Giorgio Ausiello. Prize-collecting traveling salesman and related problems. In *Handbook of Approximation Algorithms and Metaheuristics*, 2007.